

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Khoa Mạng máy tính và Truyền thông



UIT
TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

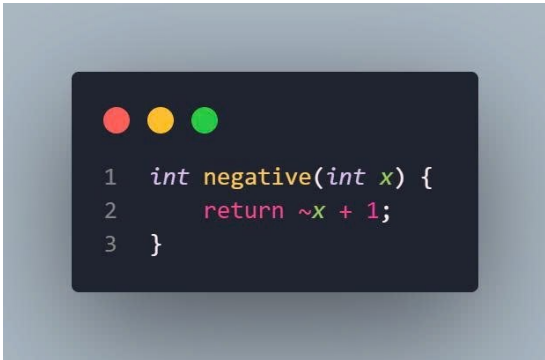
Môn học: Lập trình hệ thống

LAB 1: *Data Lab: Tính toán Bits*

Giảng viên hướng dẫn		Đỗ Hoàng Hiền
Nhóm 11	24520557	Nguyễn Minh Hoàng
	24520602	Nguyễn Duy Hưng
	22520810	Huỳnh Bảo Long (vắng buổi 1)

Thành phố Hồ Chí Minh, ngày 30 tháng 9 năm 2025

BÀI 2.1: Tính toán bit cơ bản

STT	Code & Chú thích	Chú thích
1.1	 <pre> 1 int bitAnd(int x, int y) { 2 return ~(~x ~y); 3 } </pre>	Tác dụng: Hàm bitAnd thực hiện phép toán AND giữa hai số nguyên mà không dùng trực tiếp toán tử &.
		Thực hiện: áp dụng công thức De Morgan để chuyển về dạng $\sim(\sim x \sim y)$.
1.2	 <pre> 1 int negative(int x) { 2 return ~x + 1; 3 } </pre>	Tác dụng: hàm negative đổi một số nguyên bất kỳ sang số đối của nó.
		Thực hiện: Dùng quy tắc bù 2 (ví dụ từ dương sang âm hoặc ngược lại) bằng cách lấy bù 1 rồi cộng thêm 1.
1.3	 <pre> 1 int getByte(int x, int n) { 2 int mask = 255 << (n << 3); 3 return (x & mask) >> (n << 3); 4 } 5 </pre>	Tác dụng: Hàm getByte cho phép lấy ra byte thứ n trong một số nguyên 32-bit, với $n = 0$ là byte thấp nhất (least significant byte).
		Thực hiện: Bước 1: Tạo ra một biến mặt nạ $\text{mask} = 255 \ll (n \ll 3)$. Giá trị 255 tương ứng với 1 byte có tất cả các bit bằng 1 (1111111_2). Khi dịch trái $255 \ll (n \ll 3)$ bit, ta đã đưa dãy bit này đến đúng vị trí byte cần lấy trong số x . Bước 2: biểu thức $(x \& \text{mask})$ loại bỏ tất cả các bit phía bên trái và chỉ giữ lại phần byte mong muốn. Kết quả cuối cùng được trả về chính là giá trị nguyên của byte thứ n trong số x .
1.4	 <pre> 1 int setByte(int x, int n) { 2 int mask = 255 << (n << 3); 3 return x mask; 4 } </pre>	Tác dụng: hàm setByte bật 1 byte thứ n của số nguyên x (các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái), tất cả thành bit 1.
		Thực hiện: Bước 1: Tạo một biến mặt nạ giống hàm getByte Bước 2: $\text{return } x \text{mask};$

		Phép OR () sẽ giữ nguyên các bit khác của x, nhưng ép byte ở vị trí n thành 11111111. Nghĩa là byte thứ n sau khi chạy hàm luôn được gán bằng 255, bất kể giá trị ban đầu.
1.5	 <pre> 1 int getnbit(int x, int n) { 2 int mask = 32 + (~n + 1); 3 mask = mask & ~(mask >> 31); 4 return x & (0xFFFFFFFF >> mask); 5 } </pre>	Tác dụng: Hàm getnbit trả về n bit thấp nhất (bên phải) của số nguyên x (các bit được đánh số thứ tự từ 0 đến 31 từ phải sang trái). Thực hiện: Tạo một giá trị mặt nạ (mask) gồm n bit 1 ở cuối. Nếu $n \geq 32$, thay vì tạo mask bằng dịch trái (dễ gây tràn), hàm sẽ xử lý đặc biệt để mask trở thành toàn 1 (111...111), tức giữ nguyên giá trị x. Cuối cùng, x được AND với mask để loại bỏ những bit dư thừa bên trái, chỉ còn lại n bit thấp nhất.

**** Giải thích rõ trường hợp câu 1.5 overflow ($n > 31$)**

- **Dòng 1: $\text{int mask} = 32 + (\sim n + 1)$:** Thực hiện phép tính $32 - n$ bằng bù hai ($\sim n + 1$ tương đương $-n$).

Giá trị này biểu diễn số bit còn lại khi ta chỉ muốn giữ lại n bit cuối của một số nguyên 32bit.

- **Dòng 2: $\text{mask} = \text{mask} \& \sim(\text{mask} \gg 31)$:** Biểu thức được dùng để đảm bảo mask không âm.

Phép dịch phải ($\text{mask} \gg 31$) lấy ra bit dấu: $= 0$ (nếu mask dương); $= 1$ (nếu mask âm).

Khi phủ định với toán tử $\sim$, ta thu được:

- $\text{mask} \geq 0$: toàn bộ bit 1.
- $\text{mask} < 0$: toàn bộ bit 0.

Nhờ đó, khi kết hợp với toán tử AND, mask được giữ nguyên nếu dương và thành 0 nếu âm.

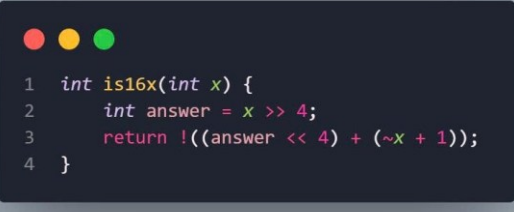
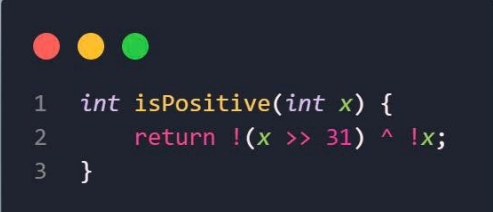
- **Dòng 3: $\text{return } x \& (0xFFFFFFFF \gg \text{mask})$:**

Biểu thức trong ngoặc tạo một dãy bit 1 có đúng n chữ số nhị phân ở cuối.

Bởi vì dịch phải $32 - n$ bit từ số có 32 bit 1 $\Rightarrow$ Sẽ còn lại đúng n bit 1.

Khi AND với x , kết quả trả về **chính là n bit thấp nhất của x .**

Bài 2.2: Các phép kiểm tra dựa trên tính toán bit

STT	Code	Kết quả thực thi
2.1	 <pre> 1 int isOpposite(int x, int y) { 2 return !(x + y); 3 } </pre>	Tác dụng: Hàm isOpposite kiểm tra xem hai số có phải là số đối nhau hay không. Thực hiện: cộng $x + y$: nếu kết quả là 0 thì nghĩa là $y = -x$, và khi đó trả về !0, ngược lại trả về !1.
2.2	 <pre> 1 int is16x(int x) { 2 int answer = x >> 4; 3 return !((answer << 4) + (~x + 1)); 4 } </pre>	Tác dụng: Hàm is16x dùng để kiểm tra xem x có chia hết cho 16 không. Thực hiện: Cách làm là dịch phải x 4 bit để thực hiện phép chia cho 16, sau đó dịch trái trở lại và so sánh với x. Nếu kết quả giống hệt thì chứng tỏ x chia hết cho 16.
	 <pre> 1 int isPositive(int x) { 2 return !(x >> 31) ^ !x; 3 } </pre>	Tác dụng: Hàm isPositive cho biết một số có phải số dương không. Thực hiện: - Dịch phải x 32 bit: lấy ra bit dấu. - Phủ định (!) giá trị đó để biết x có âm không. Đồng thời phủ định x: Nếu $x = 0 \rightarrow$ thành 1. Nếu x khác 0 $\rightarrow$ thành 0. - Sau đó XOR 2 kết quả để phân biệt trường hợp: ▪ Nếu $x < 0$: bit dấu = 1 $\rightarrow$ sau khi NOT thành 0, $x \neq 0$ nên NOT $x = 0$. Kết quả: $0 \text{ xor } 0 = 0 \rightarrow$ trả về 0 (không dương). ▪ Nếu $x > 0$: bit dấu = 0 $\rightarrow$ sau khi NOT thành 1, $x \neq 0$ nên NOT $x = 0$. Kết quả: $1 \text{ xor } 0 = 1 \rightarrow$ trả về 1 (số dương). ▪ Nếu $x = 0$: bit dấu = 0 $\rightarrow$ sau khi NOT thành 1, NOT $x = 1$. Kết quả: $1 \text{ xor } 1 = 0 \rightarrow$ trả về 0 (không dương).

```
1  int isGE2n(int x, int n) {  
2      int temp = 1 << n;  
3      int ans = x + (~temp + 1);  
4      return !(ans >> 31);  
5  }
```

Tác dụng: hàm isGE2n kiểm tra x có lớn hơn hoặc bằng 2^n hay không.

Thực hiện: Hàm này dịch trái 1 lên n vị trí để tạo giá trị 2^n , sau đó trừ đi từ x bằng cách cộng với bù 2, rồi quan sát bit dấu của kết quả.

Nếu kết quả không âm thì tức là $x \geq 2^n$.

III. Kết quả thực thi



```
1  1.1 bitAnd      Pass.
2  1.2 negative    Pass.
3  1.3 getByte     Pass.
4  1.4 setByte     Pass.
5  1.5 getnbit     Advanced Pass.
6  2.1 isOpposite  Pass.
7  2.2 is16x      Pass.
8  2.3 isPositive  Pass.
9  2.4 isGE2n     Pass.
10 -----
11 Your score: 10.0
```